



## 2.4 从正规式到词法分析器

构造词法分析器的一般方法和步骤：

1. 用正规式对模式进行描述；
2. 为每个正规式构造一个NFA，它识别正规式所表示的正规集；
3. 将构造出的NFA转换成等价的DFA，这一过程也被称为**确定化**；
4. 优化DFA，使其状态数最少，这一过程也被称为**最小化**；
5. 从优化后的DFA构造词法分析器。

问题：

我们是从DFA构造词法分析器，为何不直接从正规式构造DFA，而要先构造NFA，然后转换为DFA？

原因：希望使用机器（工具）构造词法分析器，而机器构造需要规范的算法。（词法分析器生成器Lex的工作原理）

<1>正规式→NFA：有规范的一对一的构造算法

<2> NFA确定化、DFA最小化都有统一的算法

<3> DFA→分析器：有便于记号识别的算法



## 2.4.1 从正规式到NFA

### 算法2.2 Thompson 算法

输入 字母表 $\Sigma$ 上的正规式 $r$

输出 接受 $L(r)$ 的NFA  $N$

方法 首先分解 $r$ ，然后根据下述步骤构造NFA：

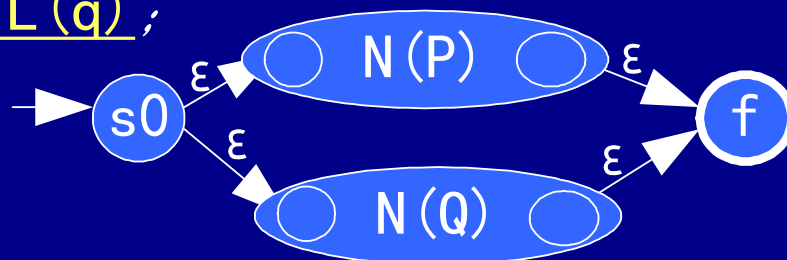
<1> 对 $\varepsilon$ ，构造NFA  $N(\varepsilon)$ 如下。其中， $s_0$ 为初态， $f$ 为终态，此NFA接受 $\{\varepsilon\}$ ；



<2> 对 $\Sigma$ 上的每个字符 $a$ ，构造NFA  $N(a)$ 如右上，它接受 $\{a\}$ ；

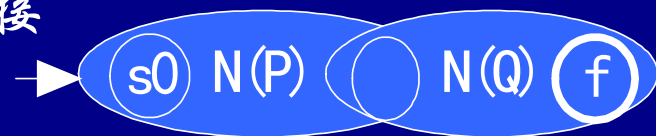
<3> 若 $N(p)$ 和 $N(q)$ 是正规式 $p$ 和 $q$ 的NFA，则

(a) 对正规式 $p|q$ ，构造NFA  $N(p|q)$ 如下。其中， $s_0$ 为初态， $f$ 为终态，此NFA接受 $L(p) \cup L(q)$ ；

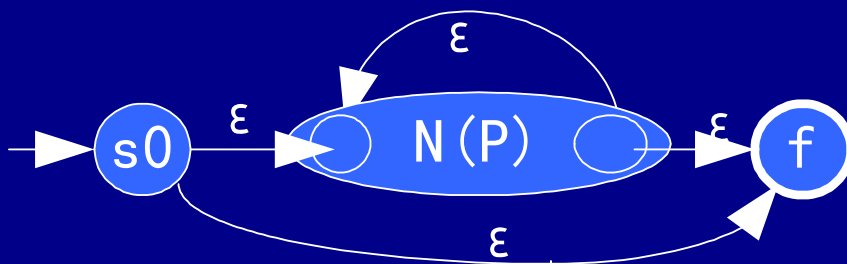




(b) 对正规式  $\underline{pq}$ , 构造NFA  $\underline{N(pq)}$  如右。其中  $s_0$  为初态,  $f$  为终态, 此NFA接受  $\underline{L(p)L(q)}$ ;



(c) 对正规式  $\underline{p^*}$ , 构造NFA  $\underline{N(p^*)}$  如右。其中,  $s_0$  为初态,  $f$  为终态, 此NFA接受  $\underline{L(p^*)}$ ;



<4> 对于正规式  $\underline{(p)}$ , 使用  $p$  本身的NFA, 不再构造新的NFA。



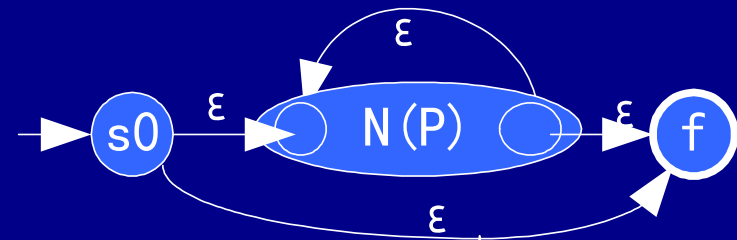
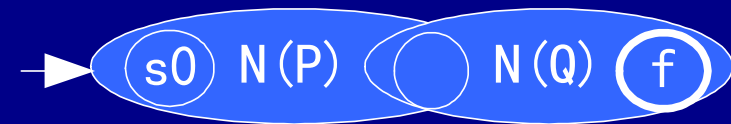
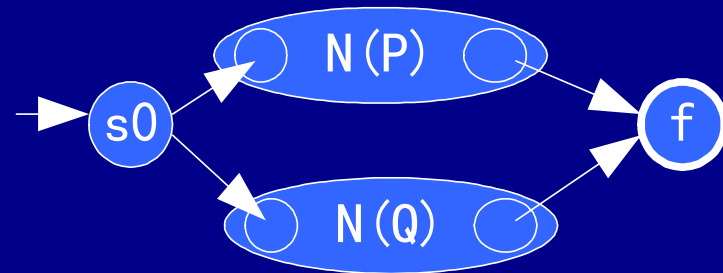
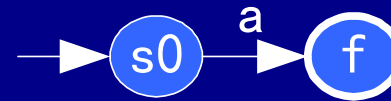


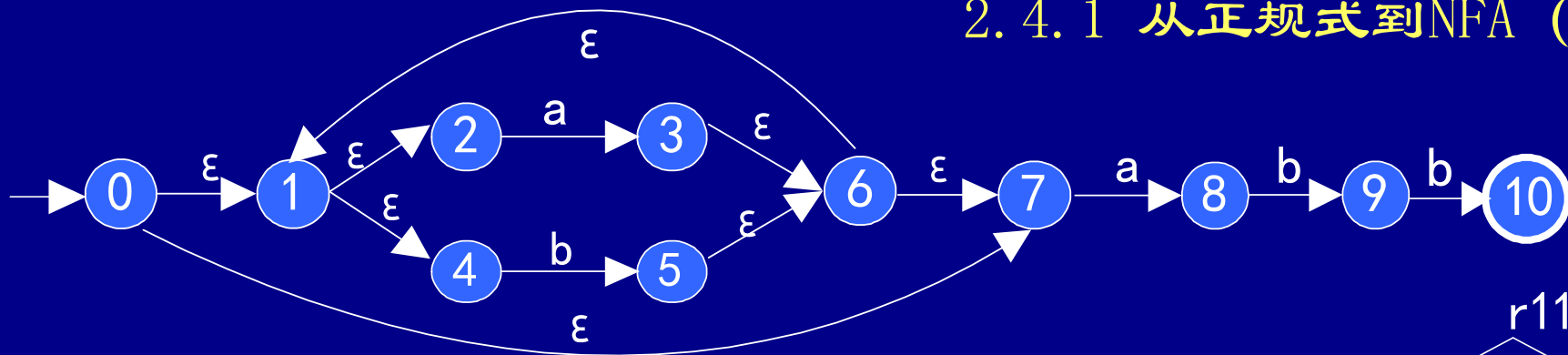
## 正规式与NFA的对应关系:

## 正规式

1.  $\varepsilon$  表示集合  $L(\varepsilon) = \{\varepsilon\}$ 2.  $a$  表示集合  $L(a) = \{a\}$ 3.  $P|Q$  表示集合  $L(P) \cup L(Q)$ 4.  $PQ$  表示集合  $L(P)L(Q)$ 5.  $P^*$  表示集合  $(L(P))^*$ 6.  $(r)$  仍然表示集合  $L(r)$ 

## NFA





例2.11 用Thompson算法构造正规式

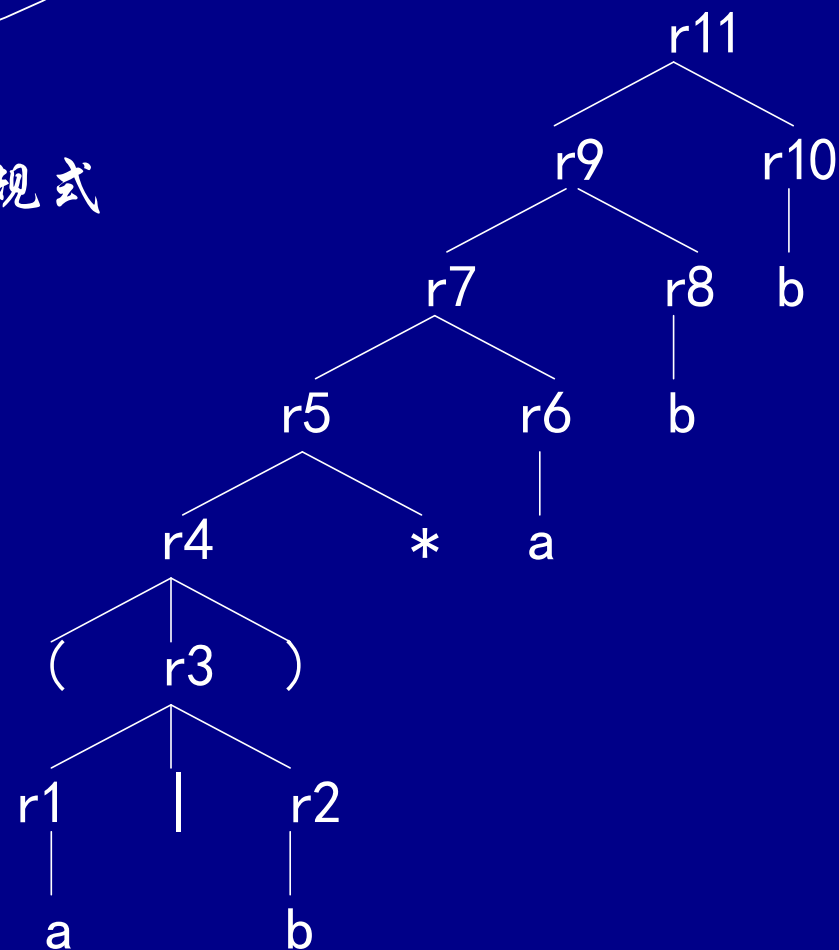
$r = (a|b)^*abb$ 的NFA  $N(r)$

<1> 分解正规式

<2> 自下而上构造NFA

强调:

- 算法的构造与正规式一一对应
- 构造一个新的NFA最多增加两个状态





## 2.4 从正规式到词法分析器

构造词法分析器的一般方法和步骤：

1. 用正规式对模式进行描述；
2. 为每个正规式构造一个NFA，它识别正规式所表示的正规集；
3. 将构造出的NFA转换成等价的DFA，这一过程也被称为**确定化**；
4. 优化DFA，使其状态数最少，这一过程也被称为**最小化**；
5. 从优化后的DFA构造词法分析器。

问题：

我们是从DFA构造词法分析器，为何不直接从正规式构造DFA，而要先构造NFA，然后转换为DFA？

原因：希望使用机器（工具）构造词法分析器，而机器构造需要规范的算法。（词法分析器生成器Lex的工作原理）

<1>正规式→NFA：有规范的一对一的构造算法

<2> NFA确定化、DFA最小化都有统一的算法

<3> DFA→分析器：有便于记号识别的算法



## 2.4.2 从NFA到DFA

### <1> NFA识别记号的“并行”方法

例2.12 从甲地到乙地，可以乘小轿车也可以骑自行车，具体路线如右图。其中c表示乘车，b表示骑自行车。现在要求从甲地到乙地，只许乘车而不许骑自行车，可行吗？

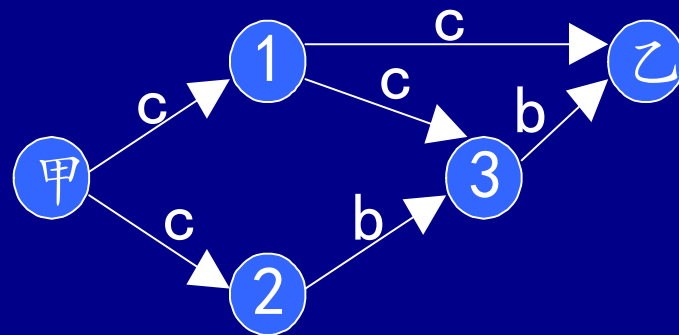
问题抽象：识别是否有从甲到乙标记为全c的路径

试探（串行）：

甲 c 2                      无路可走，退回

甲 c 1 c 3                  无路可走，退回

甲 c 1 c 乙                  到达乙地，成功



假设有足够多的小汽车，每次均到达小汽车可能到达的全体  
并行：

甲 c {1, 2} c {3, 乙}    到达乙地，成功

问题：这种并行方法为什么不需要回溯？



由于并行的方法在每试探一步时，考虑了所有的下一状态转移，因此所走的每一步都是确定的。

用NFA识别记号，并不采用串行的方法（算法不易构造，复杂度高且回溯），而是采用并行的方法，核心思想是将不确定的下一状态确定化。

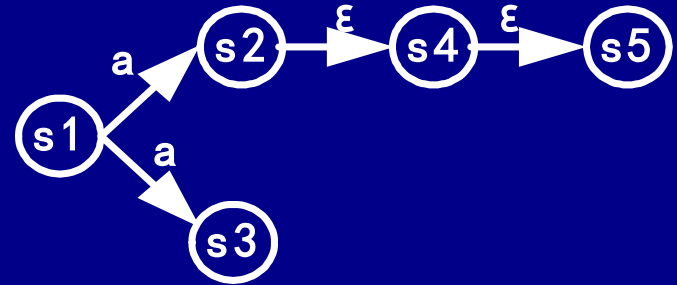
## NFA上识别记号的确定化方法

确定化的两个步骤（回顾DFA定义）

计算下一状态转移时：

<1> 消除  $\epsilon$  状态转移： $\epsilon$ -闭包(T)，**状态集T**的  $\epsilon$  闭包

<2> 消除多于一个的下一状态转移： $\text{smove}(S, a)$ ，S是一个状态集， **$a \neq \epsilon$**







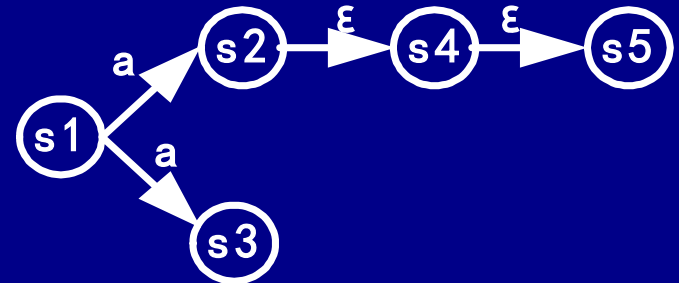
- $\text{smove}(S, a)$ : 从状态集 $S$ 中的每个状态出发, 经过标记为 $a$ 的边直接到达的下一状态全体。与 $\text{move}(s, a)$ 的唯一区别: 用状态集取代状态
- $\varepsilon$ -闭包( $T$ ): 从状态集 $T$ 中的每个状态出发, 经过若干次 $\varepsilon$ 转移到达的状态全体。

**定义2.6** 状态集 $T$ 的 $\varepsilon$ -闭包( $T$ )是一个状态集, 且满足:

- (1)  $T$ 中所有状态属于 $\varepsilon$ -闭包( $T$ );
- (2) 如果 $t$ 属于 $\varepsilon$ -闭包( $T$ )且 $\text{move}(t, \varepsilon) = u$ , 则 $u$ 属于 $\varepsilon$ -闭包( $T$ );
- (3) 再无其他状态属于 $\varepsilon$ -闭包( $T$ )。 ■

根据定义,  $\varepsilon$ -闭包( $\{s_2\}$ )应包括:

1.  $s_2$ 自身       $\{s_2\}$       (1)
2.  $s_4$        $\{s_2, s_4\}$       (2)
3.  $s_5$        $\{s_2, s_4, s_5\}$       (2)



算法



## 算法2.4 求 $\varepsilon$ -闭包

输入 状态集  $T$ 。

输出 状态集  $T$  的  $\varepsilon$ -闭包

方法 用下边的函数计算  $\varepsilon$ -闭包

function  $\varepsilon$ -闭包( $T$ ) is

begin

for  $T$  中每个状态  $t$

loop 加入  $t$  到  $U$ ; push( $t$ );

end loop;

while 栈不空

loop pop( $t$ );

for 每个  $u = \text{move}(t, \varepsilon)$

loop if  $u$  不在  $U$  中 then 加入  $u$  到  $U$ ; push( $u$ ); end if;

end loop;

end loop;

**return**  $U$ ;

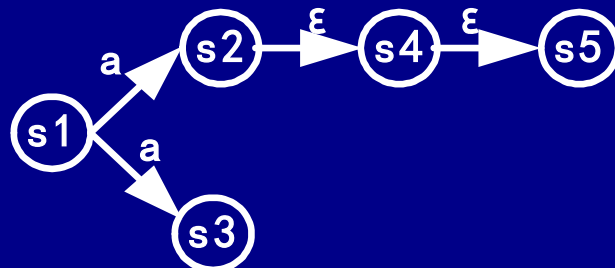
end  $\varepsilon$ -闭包;

两个数据结构:

闭包  $U$  和模拟递归的 stack

用算法计算  $\varepsilon$ -闭包( $\{s_2\}$ ):

| $U$                    | stack |
|------------------------|-------|
| 1. $\{s_2\}$           | $s_2$ |
| 2. $\{s_2, s_4\}$      | $s_4$ |
| 3. $\{s_2, s_4, s_5\}$ | $s_5$ |
| 4. $\{s_2, s_4, s_5\}$ |       |





## 算法2.3 模拟NFA

## 2.4.2 从NFA到DFA (续3)

输入 NFA  $N$ ,  $x$  (eof),  $s_0$  (NFA初态),  $F$  (NFA终态集)

输出 若 $N$ 接受 $x$ , 回答“yes”, 否则“no”

方法 用下边的过程对 $x$ 进行识别。 $S$ 是一个状态的集合

```

 $S := \varepsilon$ -闭包( $\{s_0\}$ );           -- 所有可能初态的集合
 $a := \text{nextchar};$ 
while  $a \neq \text{eof}$  loop
     $S := \varepsilon$ -闭包( $\text{smove}(S, a)$ ); -- 所有下一状态的集合
     $a := \text{nextchar};$ 
end loop;
if  $S \cap F \neq \Phi$  then return “yes”; else return “no”;
end if;
    
```

与算法2.1的三点区别: 模拟DFA

模拟NFA

1. 开始 初态( $s_0$ )

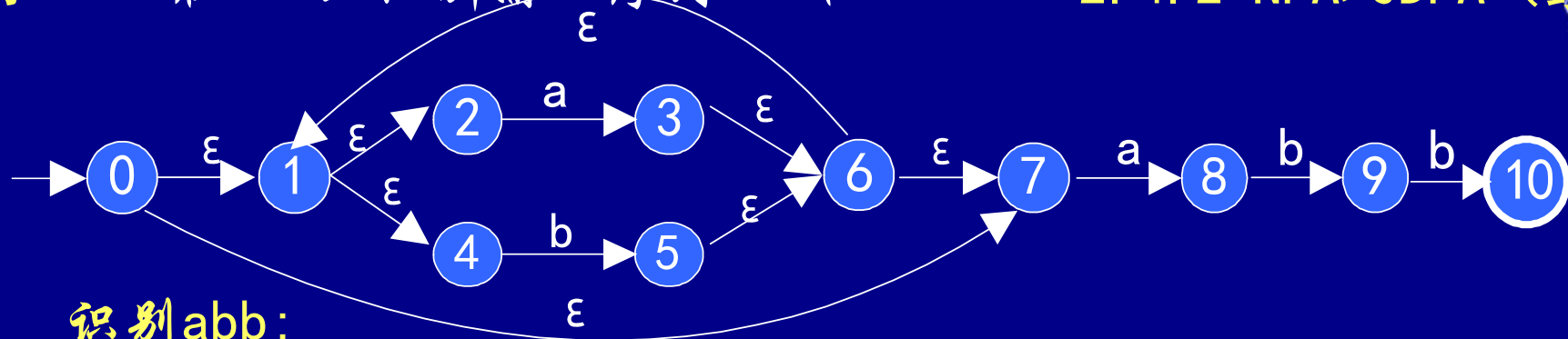
初态集( $S$ )

2. 下一状态转移 下一状态

下一状态集

3. 结束  $s$  is in  $F$

$S \cap F \neq \Phi$



识别abb:

- 1 计算初态集:  $\epsilon$ -闭包( $\{0\}$ ) =  $\{0, 1, 2, 4, 7\}$ , A
- 2 从A出发经a到达:  $\epsilon$ -闭包( $\text{smove}(A, a)$ ) =  $\{3, 8, 6, 7, 1, 2, 4\}$ , B
- 3 从B出发经b到达:  $\epsilon$ -闭包( $\text{smove}(B, b)$ ) =  $\{5, 9, 6, 7, 1, 2, 4\}$ , C
- 4 从C出发经b到达:  $\epsilon$ -闭包( $\text{smove}(C, b)$ ) =  $\{5, \underline{10}, 6, 7, 1, 2, 4\}$ , D
- 5 结束且 $D \cap \{10\} = \{10\}$ , 接受。识别的路径为: A a B b C b D

0  $\epsilon^*$  A a  $\epsilon^*$  B b  $\epsilon^*$  C b  $\epsilon^*$  D 路径上的标记:  $\epsilon^* a \epsilon^* b \epsilon^* b \epsilon^* = abb$

识别abab:

- 1 初态集:  $\epsilon$ -闭包( $s_0$ ) =  $\{0, 1, 2, 4, 7\}$ , A
- 2 从A出发经a到达:  $\epsilon$ -闭包( $\text{smove}(A, a)$ ) =  $\{3, 8, 6, 7, 1, 2, 4\}$ , B
- 3 从B出发经b到达:  $\epsilon$ -闭包( $\text{smove}(B, b)$ ) =  $\{5, 9, 6, 7, 1, 2, 4\}$ , C
- 4 从C出发经a到达:  $\epsilon$ -闭包( $\text{smove}(C, a)$ ) =  $\{3, 8, 6, 7, 1, 2, 4\}$ , B
- 5 从B出发经b到达:  $\epsilon$ -闭包( $\text{smove}(B, b)$ ) =  $\{5, 9, 6, 7, 1, 2, 4\}$ , C

识别路径为: A a B b C a B b C。由于 $C \cap \{10\} = \emptyset$ , 所以不接受

## <2> “子集法”构造DFA

## 2.4.2 NFA到DFA (续5)



“并行”模拟NFA的弱点：每次动态计算下一状态转移的集合，效率低。

改进方法：将NFA上的全部路径均确定化并且记录下来，得到与NFA等价的DFA。

回顾从甲地到乙地的路径，它的数学模型实质上是一个NFA（右上）。

可以找到一个等价的DFA（右下）。

它们识别的路径均是：

cc

ccb

cbb

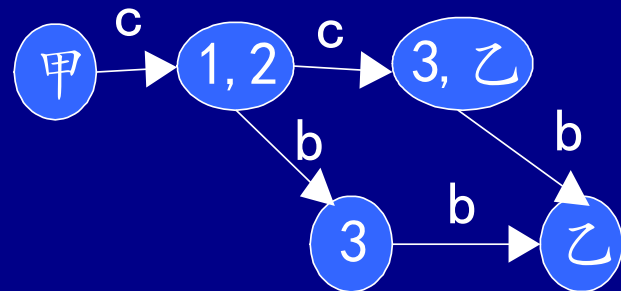
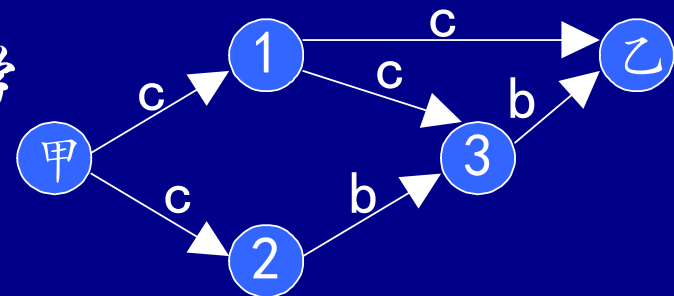
例2.14 用DFA识别cc和cbc：

甲 c {1, 2} c {3, 乙}, 接受

甲 c {1, 2} b {3} c ?, 不接受

优点：

1. 消除了不确定性（将NFA的下一状态集合合并为一个状态）
2. 无需动态计算状态集合（针对模拟NFA的算法）



## 算法2.5 从NFA构造DFA (子集法)

## 2.4.2 NFA到DFA (续6)



输入 NFA  $N$

输出 等价的DFA  $D$ 。初态是  $\varepsilon$ -闭包( $\{s_0\}$ )，终态是含有NFA终态的状态集合 两个数据结构:  $Dstates$ (状态),  $Dtran$ (状态转移)

方法 用下述过程构造DFA:

$Dstates = \{ \varepsilon\text{-闭包}(\{s_0\}) \}$ ; //  $Dstates$ 中仅有一个状态且未标记

while  $Dstates$ 有尚未标记的状态 $T$

loop 标记 $T$ ;

for 每一个输入字符 $a$

loop  $U := \varepsilon\text{-闭包}(\text{move}(T, a))$ ;

if  $U$ 非空

then  $Dtran[T, a] := U$ ;

if  $U$ 不在 $Dstates$ 中

then  $U$ 作为尚未标记的状态加入 $Dstates$ ;

end if;

end if;

end loop;

end loop;

与算法2.3比较:  
记录了所有状态  
与状态转移





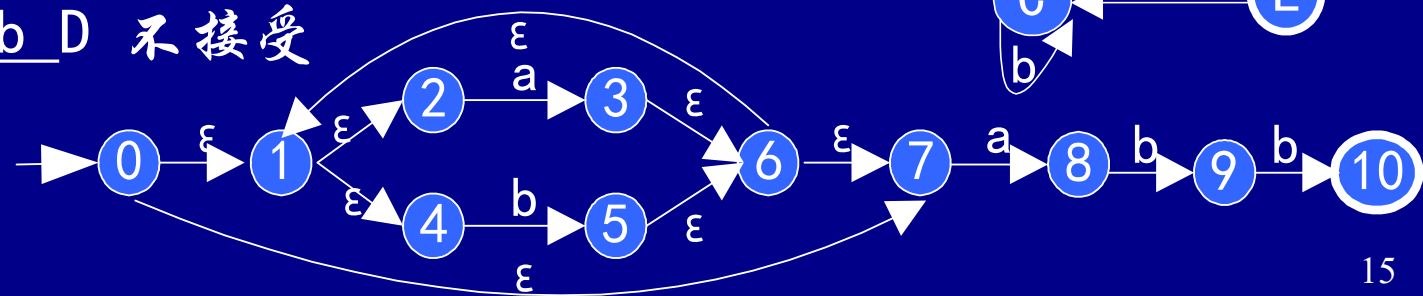
例2.15 用算法2.5构造  $(a|b)^*abb$  的DFA

- $\epsilon$ -闭包  $(\{0\}) = \{0, 1, 2, 4, 7\}^*$
- $\epsilon$ -闭包  $(\text{smove}(A, a)) = \{3, 8, 6, 7, 1, 2, 4\}^*$
- $\epsilon$ -闭包  $(\text{smove}(A, b)) = \{5, 6, 7, 1, 2, 4\}^*$
- $\epsilon$ -闭包  $(\text{smove}(B, a)) = \{3, 8, 6, 7, 1, 2, 4\}$
- $\epsilon$ -闭包  $(\text{smove}(B, b)) = \{5, 9, 6, 7, 1, 2, 4\}^*$
- $\epsilon$ -闭包  $(\text{smove}(C, a)) = \{3, 8, 6, 7, 1, 2, 4\}$
- $\epsilon$ -闭包  $(\text{smove}(C, b)) = \{5, 6, 7, 1, 2, 4\}$
- $\epsilon$ -闭包  $(\text{smove}(D, a)) = \{3, 8, 6, 7, 1, 2, 4\}$
- $\epsilon$ -闭包  $(\text{smove}(D, b)) = \{5, 10, 6, 7, 1, 2, 4\}^*$
- $\epsilon$ -闭包  $(\text{smove}(E, a)) = \{3, 8, 6, 7, 1, 2, 4\}$
- $\epsilon$ -闭包  $(\text{smove}(E, b)) = \{5, 6, 7, 1, 2, 4\}$

识别  $abb$  和  $abab$ :

A a B b D b E      接受

A a B b D a B b D      不接受

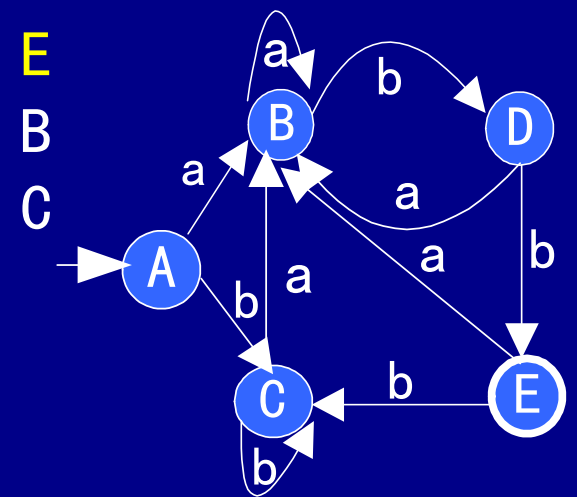
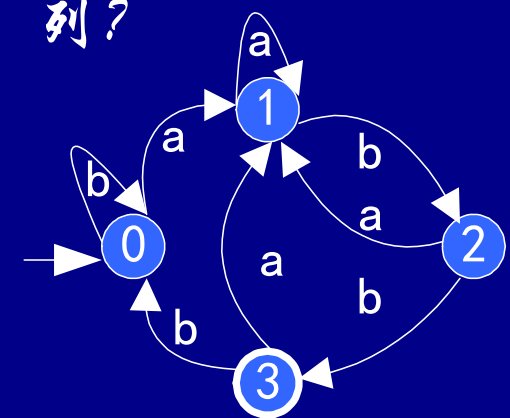


2.4.2 NFA到DFA (续7)

A      问题: 用哪个

B      DFA识别输入序

C      列?



## 2.4.3 最小化DFA

正规式—NFA—DFA—最小DFA



引入一个“可区分”的概念：

**定义2.7** 对于DFA中的任何两个状态 $t$ 和 $s$ ，若从一状态出发接受输入字符串 $w$ ，而从另一状态出发不接受 $w$ ，则称 $w$ 区分状态 $t$ 和 $s$ 。如果存在某个能够区分状态 $s$ 和状态 $t$ 的串，那么它们就是可区分的。 ■

**反方向思考定义2.7：**

设想任何输入序列 $w$ 对 $s$ 和 $t$ 均是不可区分的，则说明从 $s$ 出发和从 $t$ 出发，分析任何输入序列 $w$ 均得到相同结果。

因此， $s$ 和 $t$ 可以合并成一个状态。

**最小化DFA的本质：**

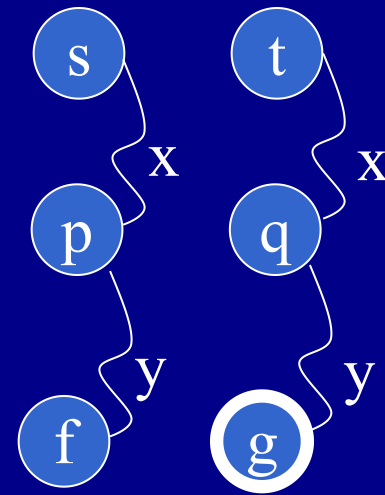
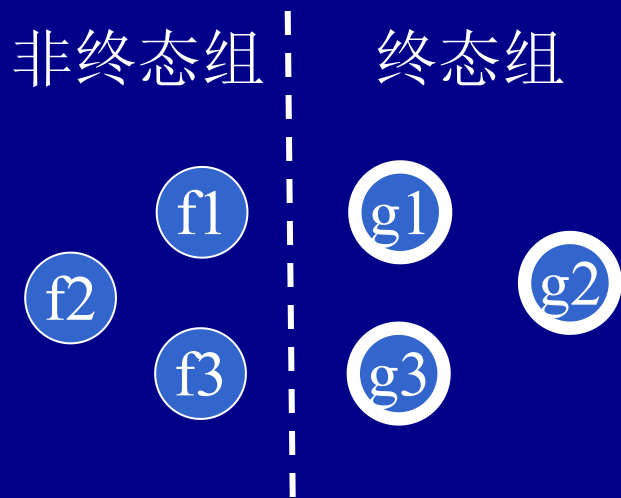
将DFA中的状态分成不同的组，使得同一组中的状态之间不可区分，不同组的状态之间是可区分的。





给定一个DFA,

- DFA的终态和非终态是可区分的;
- 如果分别从s和t出发, 沿着标记为x的路径到达的两个状态是可区分的, 则s和t也是可区分的。



基于已区分的非终态组和终态组, 进一步分裂非终态组。  
例如, 若对于同一字符a,  $\text{move}(f2, a) = f1$   $\text{move}(f3, a) = g1$ ,  
因为f1和g1可区分, 所以f2和f3可区分。

终态组的分裂方法类似。



## 算法2.6 最小化DFA的状态数

输入 DFA  $D = \{S, \Sigma, \text{move}, s_0, F\}$ 。

输出 等价的  $D' = \{S', \Sigma, \text{move}', s_0', F'\}$  ( $D'$  状态数最少)

方法 执行如下步骤:

1. 初始划分  $\Pi = \{S-F, F\}$ ;  $\Pi_{\text{new}} = \Pi$ 。F是终态集, S-F是非终态集,
2. 应用下述过程构造新的划分  $\Pi_{\text{new}}$ :  
 for  $\Pi$  的每一个组G  
   loop 划分G, G的两个状态s和t在同一组中的充要条件是:  
     
$$\forall a \in \Sigma \forall G_i \in \Pi. (\text{move}(s, a) \in G_i \leftrightarrow \text{move}(t, a) \in G_i)$$
  
     用新划分的组替代G, 形成新的划分  $\Pi_{\text{new}}$ ;  
   end loop;
3. 若  $\Pi_{\text{new}} = \Pi$ , 令  $\Pi_{\text{final}} = \Pi$ , 转4; 否则令  $\Pi = \Pi_{\text{new}}$  并重复步骤2;

对充要条件的解释: 对于任意字母表字符a,

$\text{move}(s, a)$  和  $\text{move}(t, a)$  都在  $\Pi$  的相同分组中, 或者都没有下一状态  $q_8$



## 逻辑公式含义

s和t被分在同一个组里，**当且仅当**对于 $\Sigma$ 中的任意字符a，都能使以下两个条件之一成立：

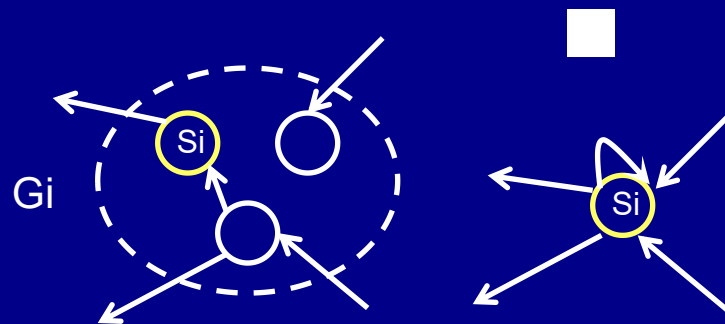
- (1) s和t遇a时都没有下一状态；
- (2) s和t遇a时都有下一状态并且两个下一状态在当前划分 $\Pi$ 的同一组中。

s和t分开，**当且仅当**存在 $\Sigma$ 中的某个字符a，能使以下两个条件之一成立：

- (1) s和t遇a时一个有下一状态，另一个没有下一状态；
- (2) s和t遇a时都有下一状态并且两个下一状态在当前划分 $\Pi$ 的不同组中。



4. **选代表并修改状态转移**: 在 $\Pi_{\text{final}}$ 每个组 $G_i$ 中选一个代表 $s_i'$ , 使得 $D$ 中从 $G_i$ 所有状态出发的状态转移在 $D'$ 中均从 $s_i'$ 出发,  $D$ 中所有转向 $G_i$ 中的状态转移在 $D'$ 中均转向 $s_i'$ ;
- 初态**: 含有 $D$ 中 $s_0$ 的状态组 $G_0$ 的代表 $s_0'$ , 称为 $D'$ 的初态;
- 终态**:  $D$ 中所有含 $F$ 中状态的 $G_j$ 的代表 $s_j'$ 构成 $D'$ 的终态集 $F'$ ;
5. **删除死状态** (即不是终态且对所有输入字符均转向其自身), 和从初态不可到达的状态。

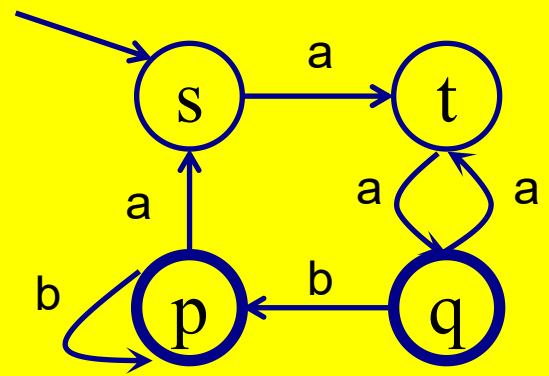


### 最小化DFA算法的主要步骤

1. 初始划分: 终态与非终态;
2. 利用可区分概念, 反复分裂划分中的组 $G_i$ , 直到不可再分裂;
3. 由最终划分构造 $D'$ , 关键是选代表和修改状态转移;
4. 消除可能的死状态和不可达状态。



DFA  $D=(S, \Sigma, \text{move}, s, F)$



1、  $\Pi=\{G1=\{s, t\}, G2=\{p, q\}\}$

G1:  
 $\text{move}(s,a)=t \in G1$   
 $\text{move}(t,a)=q \in G2$  } a可以区分s和t,  
 分裂G1为G11和G12

G2:  
 $\text{move}(p,a)=s \in G1$   
 $\text{move}(q,a)=t \in G1$  } a不能区分p和q  
 $\text{move}(p,b)=p \in G2$   
 $\text{move}(q,b)=p \in G2$  } b不能区分p和q

$\Pi_{\text{new}}=\{G11=\{s\}, G12=\{t\}, G2=\{p, q\}\}$

2、  $\Pi=\{G11=\{s\}, G12=\{t\}, G2=\{p, q\}\}$

G11和G12都只含一个状态，所以不会再分裂；  
 G2:

$\text{move}(p,a)=s \in G11$   
 $\text{move}(q,a)=t \in G12$  } a可以区分p和q,  
 分裂G2为G21和G22  
 $\text{move}(p,b)=p \in G2$   
 $\text{move}(q,b)=p \in G2$  } b不能区分p和q

$\Pi_{\text{new}}=\{G11=\{s\}, G12=\{t\}, G21=\{p\}, G22=\{q\}\}$

3、  $\Pi=\{G11=\{s\}, G12=\{t\}, G21=\{p\}, G22=\{q\}\}$

每个分组都只含一个状态，  
 所以不会再分裂；

$\Pi_{\text{new}}=\{G11=\{s\}, G12=\{t\}, G21=\{p\}, G22=\{q\}\}$

$= \Pi$   
 $= \Pi_{\text{final}}$



## 例2.17 用算法2.6化简DFA

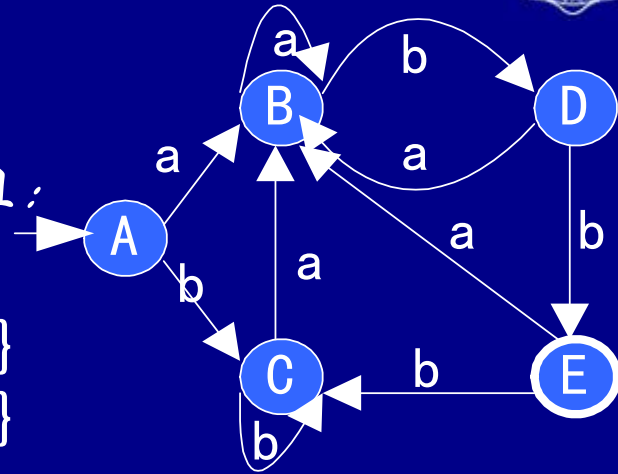
1. 初始化划分  $\Pi_1 = \{ABCD, E\}$

2. 根据算法中步骤2, 反复分裂划分中的组:

①  $\because m(D, b) = E \therefore \Pi_2 = \{ABC, D, E\}$

②  $\because m(B, b) = D \therefore \Pi_3 = \{AC, B, D, E\}$

③  $\Pi_3?$  于是:  $\Pi_{final} = \{AC, B, D, E\}$



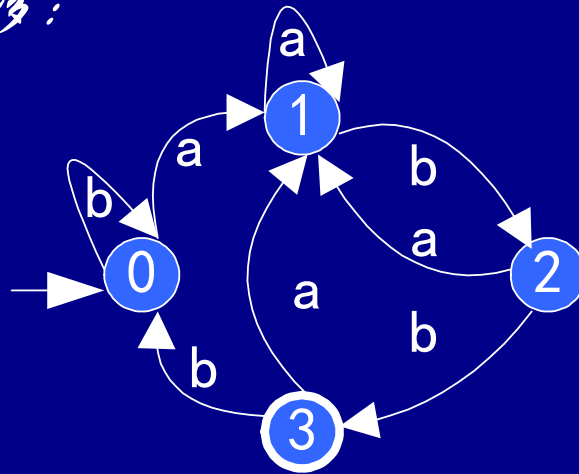
3. 根据  $\Pi_{final}$  构造  $D'$ :

① 选代表, 用A代表AC组;

② 修改状态转移:

用0、1、2、3

代替A、B、D、E;



$m(A, a) = B, m(A, b) = C$

$m(B, a) = B, m(B, b) = D$

$m(C, a) = B, m(C, b) = C$

$m(D, a) = B, m(D, b) = E$

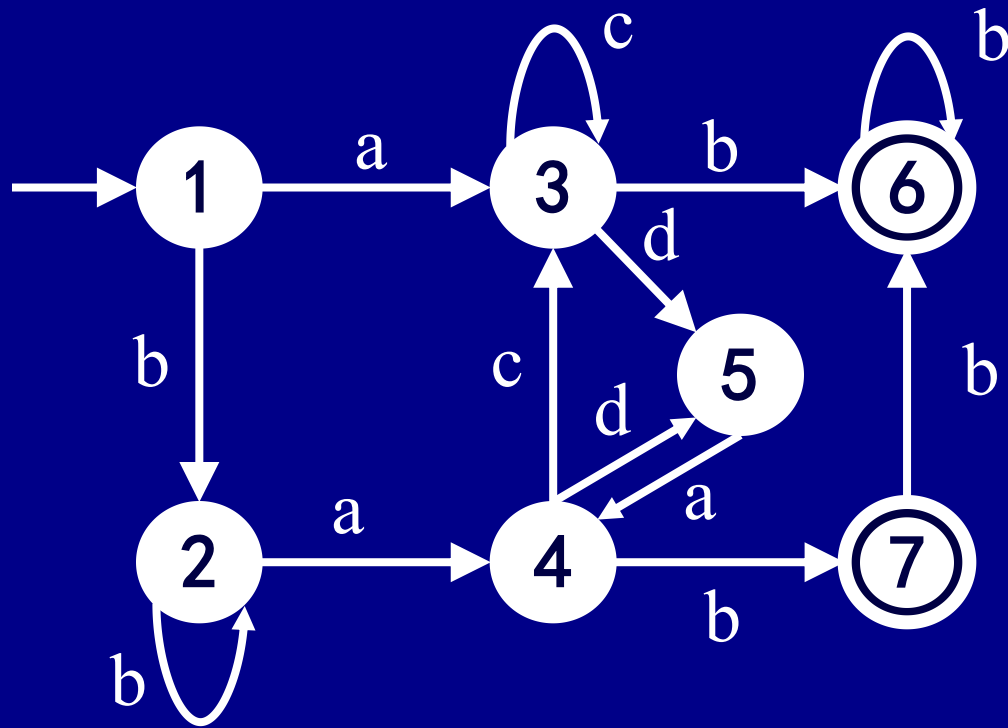
$m(E, a) = B, m(E, b) = C$

$m(A, a) = B, m(A, b) = A$

$m(B, a) = B, m(B, b) = D$

$m(D, a) = B, m(D, b) = E$

$m(E, a) = B, m(E, b) = A$



$\text{move}(1,a)=3$   
 $\text{move}(2,a)=4$   
 $\text{move}(3,a)=?$   
 $\text{move}(4,a)=?$   
 $\text{move}(5,a)=4$

$\text{move}(1,b)=2$   
 $\text{move}(2,b)=2$   
 $\text{move}(3,b)=6$   
 $\text{move}(4,b)=7$   
 $\text{move}(5,b)=?$

$\text{move}(1,c)=?$   
 $\text{move}(2,c)=?$   
 $\text{move}(3,c)=3$   
 $\text{move}(4,c)=3$   
 $\text{move}(5,c)=?$

$\text{move}(1,d)=?$   
 $\text{move}(2,d)=?$   
 $\text{move}(3,d)=5$   
 $\text{move}(4,d)=5$   
 $\text{move}(5,d)=?$

$\text{move}(6,a)=?$   
 $\text{move}(7,a)=?$

$\text{move}(6,b)=6$   
 $\text{move}(7,b)=6$

$\text{move}(6,c)=?$   
 $\text{move}(7,c)=?$

$\text{move}(6,d)=?$   
 $\text{move}(7,d)=?$



- 初始划分  $\Pi = \{G1 = \{1, 2, 3, 4, 5\}, G2 = \{6, 7\}\}$

– G1

- a 可以将 G1 分成两组:  $\{1, 2, 5\}, \{3, 4\}$
- b 可以将 G1 分成三组:  $\{1, 2\}, \{3, 4\}, \{5\}$
- c 可以将 G1 分成两组:  $\{1, 2, 5\}, \{3, 4\}$
- d 可以将 G1 分成两组:  $\{1, 2, 5\}, \{3, 4\}$

– G2

- a 不能区分 6 和 7
- b 不能区分 6 和 7
- c 不能区分 6 和 7
- d 不能区分 6 和 7

|             |             |             |             |
|-------------|-------------|-------------|-------------|
| move(1,a)=3 | move(1,b)=2 | move(1,c)=? | move(1,d)=? |
| move(2,a)=4 | move(2,b)=2 | move(2,c)=? | move(2,d)=? |
| move(3,a)=? | move(3,b)=6 | move(3,c)=3 | move(3,d)=5 |
| move(4,a)=? | move(4,b)=7 | move(4,c)=3 | move(4,d)=5 |
| move(5,a)=4 | move(5,b)=? | move(5,c)=? | move(5,d)=? |
| move(6,a)=? | move(6,b)=6 | move(6,c)=? | move(6,d)=? |
| move(7,a)=? | move(7,b)=6 | move(7,c)=? | move(7,d)=? |

$\Pi_{\text{new}} = \{G11 = \{1, 2\}, G12 = \{3, 4\}, G13 = \{5\}, G2 = \{6, 7\}\}$





- 当前划分  $\Pi = \{G11=\{1,2\}, G12=\{3,4\}, G13=\{5\}, G2=\{6,7\}\}$

- G11

- a不能区分1和2
- b不能区分1和2
- c不能区分1和2
- d不能区分1和2

- G12

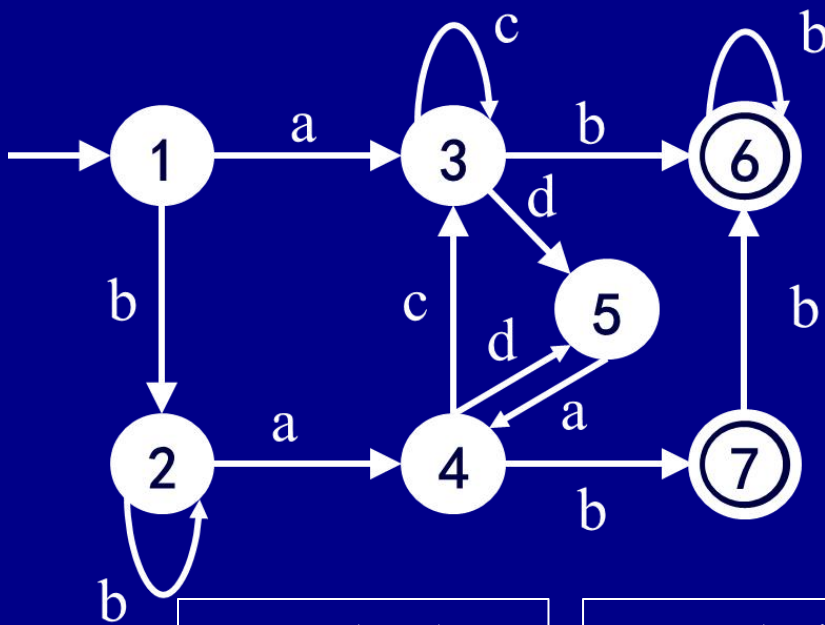
- a不能区分3和4
- b不能区分3和4
- c不能区分3和4
- d不能区分3和4

- G13不可再分裂

- G2: a,b,c,d不能区分6和7

|             |             |             |             |
|-------------|-------------|-------------|-------------|
| move(1,a)=3 | move(1,b)=2 | move(1,c)=? | move(1,d)=? |
| move(2,a)=4 | move(2,b)=2 | move(2,c)=? | move(2,d)=? |
| move(3,a)=? | move(3,b)=6 | move(3,c)=3 | move(3,d)=5 |
| move(4,a)=? | move(4,b)=7 | move(4,c)=3 | move(4,d)=5 |
| move(5,a)=4 | move(5,b)=? | move(5,c)=? | move(5,d)=? |
|             |             |             |             |
| move(6,a)=? | move(6,b)=6 | move(6,c)=? | move(6,d)=? |
| move(7,a)=? | move(7,b)=6 | move(7,c)=? | move(7,d)=? |

$\Pi_{\text{new}} = \Pi$ , 所以  $\Pi_{\text{final}} = \Pi$ 。



$\Pi_{\text{final}} = \{G11 = \{1, 2\}, G12 = \{3, 4\}, G13 = \{5\}, G2 = \{6, 7\}\}$

选1作为G11的代表, 3作为G12的代表, 5作为G13的代表, 6作为G2的代表。修改状态转移。

move(1,a)=3  
move(2,a)=4

move(3,a)=?  
move(4,a)=?

move(5,a)=3

move(6,a)=?  
move(7,a)=?

move(1,b)=1  
move(2,b)=2

move(3,b)=6  
move(4,b)=7

move(5,b)=?

move(6,b)=6  
move(7,b)=6

move(1,c)=?  
move(2,c)=?

move(3,c)=3  
move(4,c)=3

move(5,c)=?

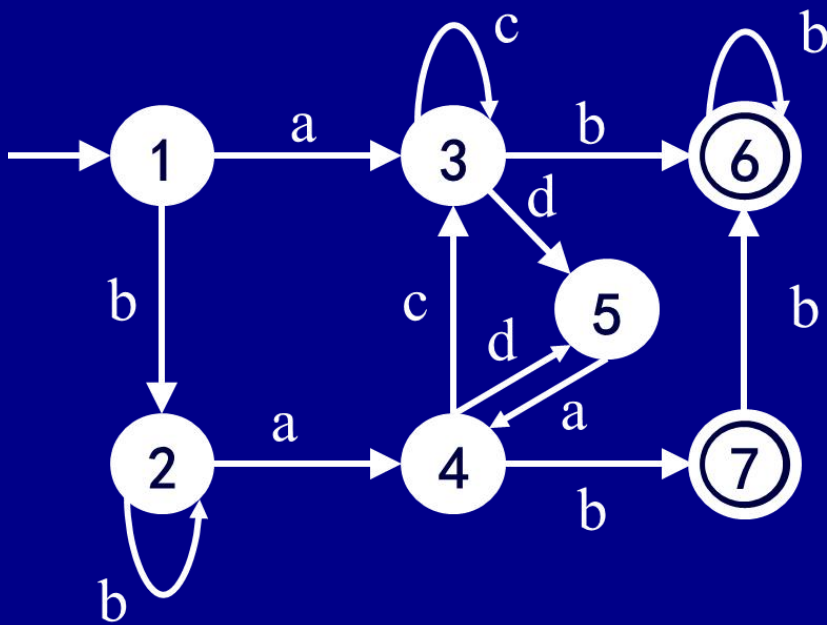
move(6,c)=?  
move(7,c)=?

move(1,d)=?  
move(2,d)=?

move(3,d)=5  
move(4,d)=5

move(5,d)=?

move(6,d)=?  
move(7,d)=?



$\Pi_{\text{final}} = \{G11 = \{1, 2\}, G12 = \{3, 4\}, G13 = \{5\}, G2 = \{6, 7\}\}$

选1作为G11的代表, 3作为G12的代表, 5作为G13的代表, 6作为G2的代表。修改状态转移。

move(1,a)=3  
move(2,a)=4

move(3,a)=?  
move(4,a)=?

move(5,a)=3

move(6,a)=?  
move(7,a)=?

move(1,b)=1  
move(2,b)=2

move(3,b)=6  
move(4,b)=7

move(5,b)=?

move(6,b)=6  
move(7,b)=6

move(1,c)=?  
move(2,c)=?

move(3,c)=3  
move(4,c)=3

move(5,c)=?

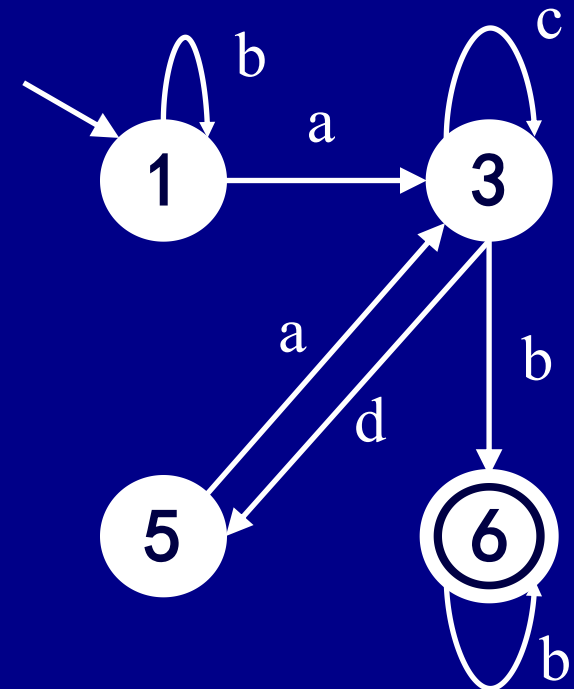
move(6,c)=?  
move(7,c)=?

move(1,d)=?  
move(2,d)=?

move(3,d)=5  
move(4,d)=5

move(5,d)=?

move(6,d)=?  
move(7,d)=?





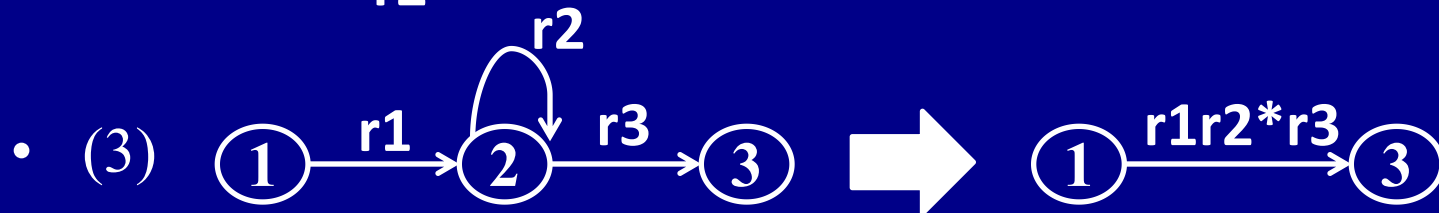
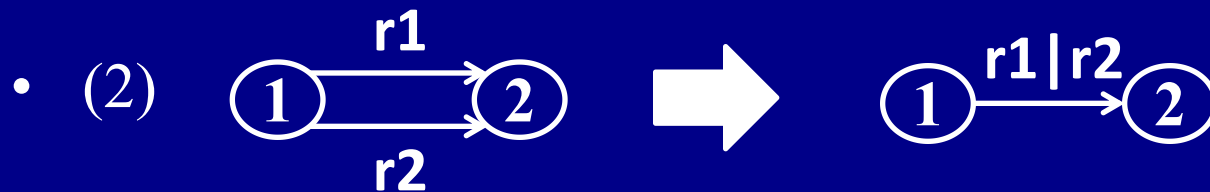
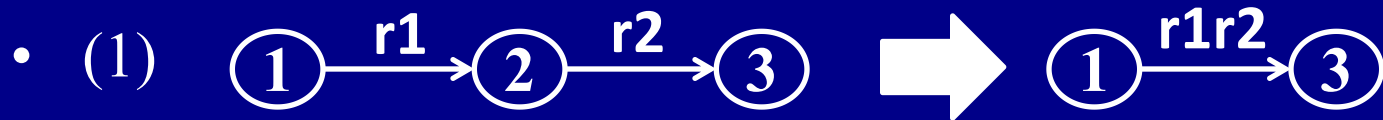
## 由NFA构造正规式

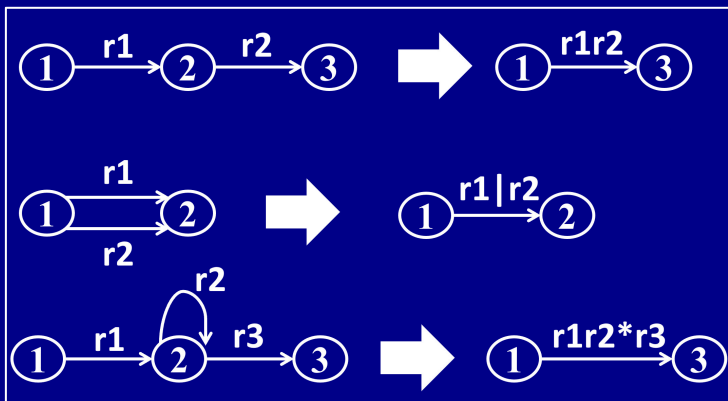
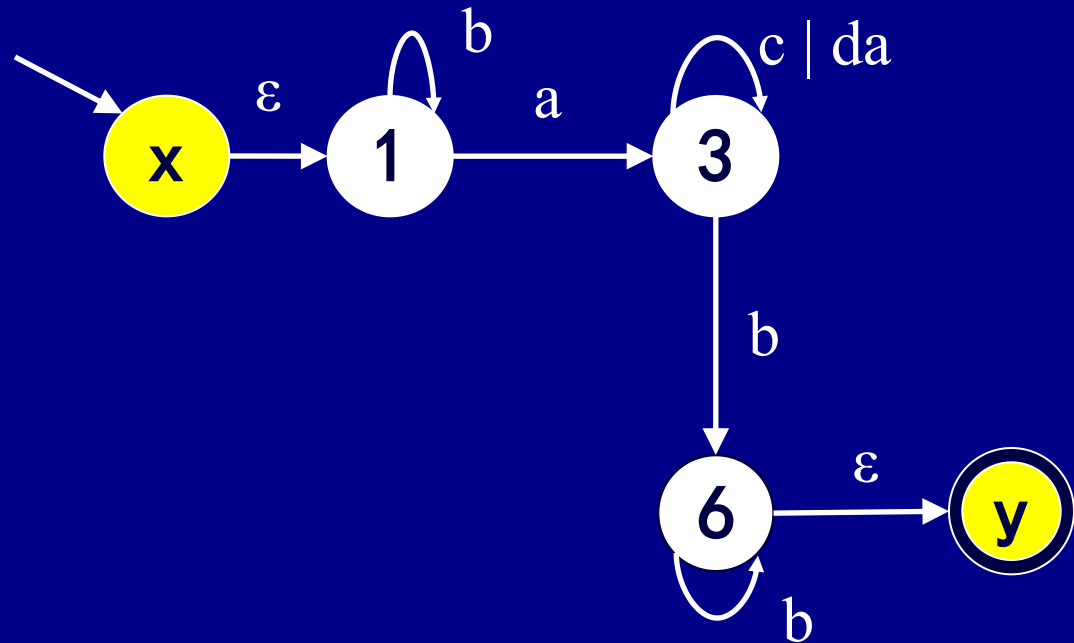
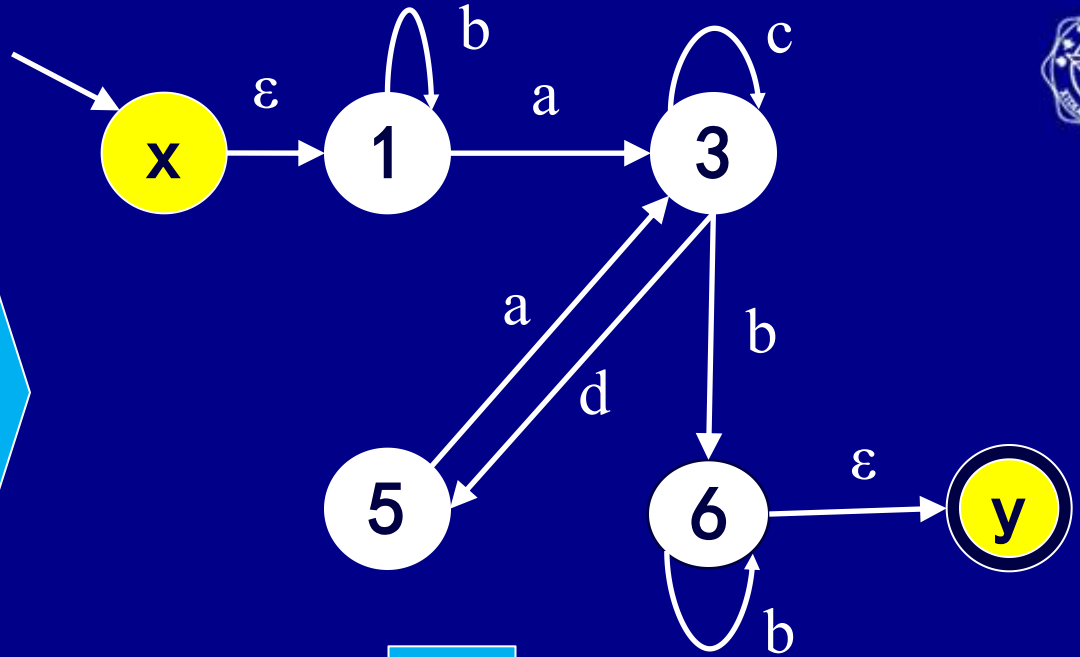
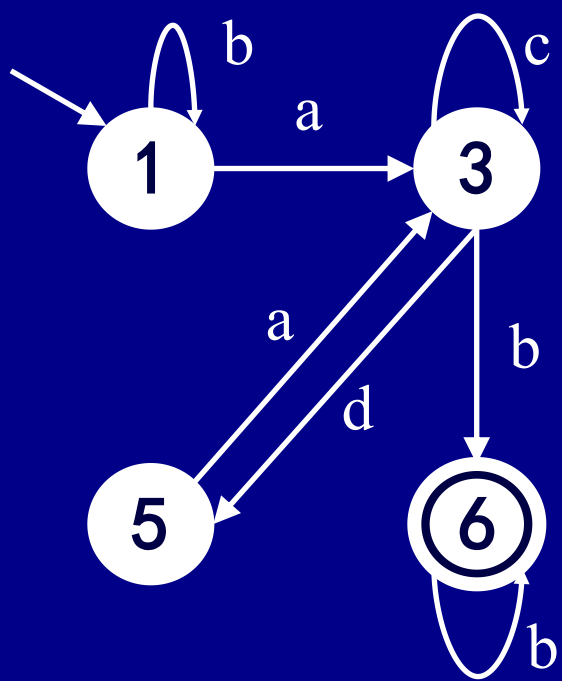
- 将状态转换图的概念拓广，令每条边可用一个正规式作标记。
- 第一步，在M的状态转换图上加进两个结点，一个为x结点，一个为y结点。  
从x结点用 $\epsilon$ 边连接到M的初态结点。  
从M的所有终态结点用 $\epsilon$ 边连接到y结点。  
形成一个与M等价的M'，M'只有一个初态x和一个终态y。

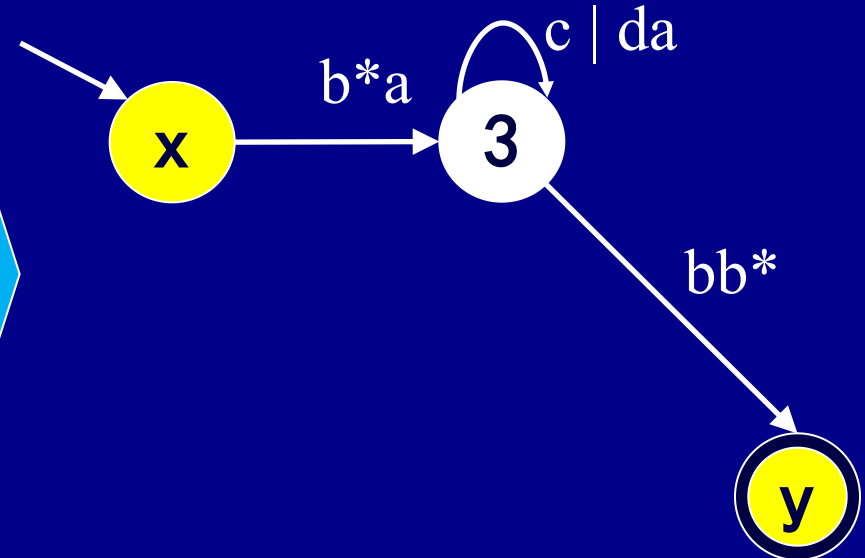
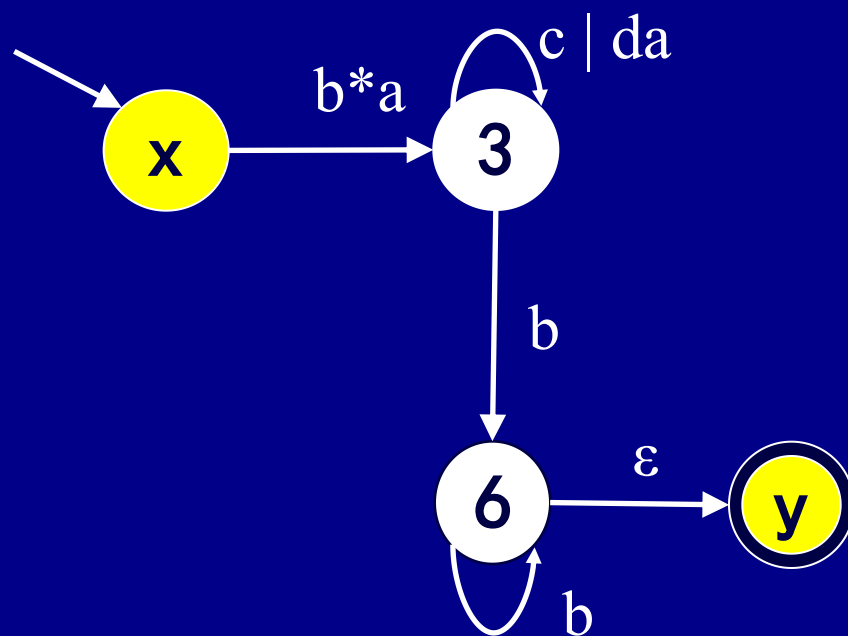
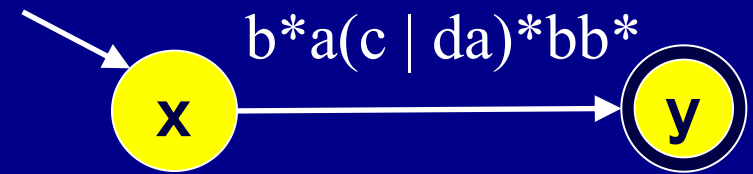
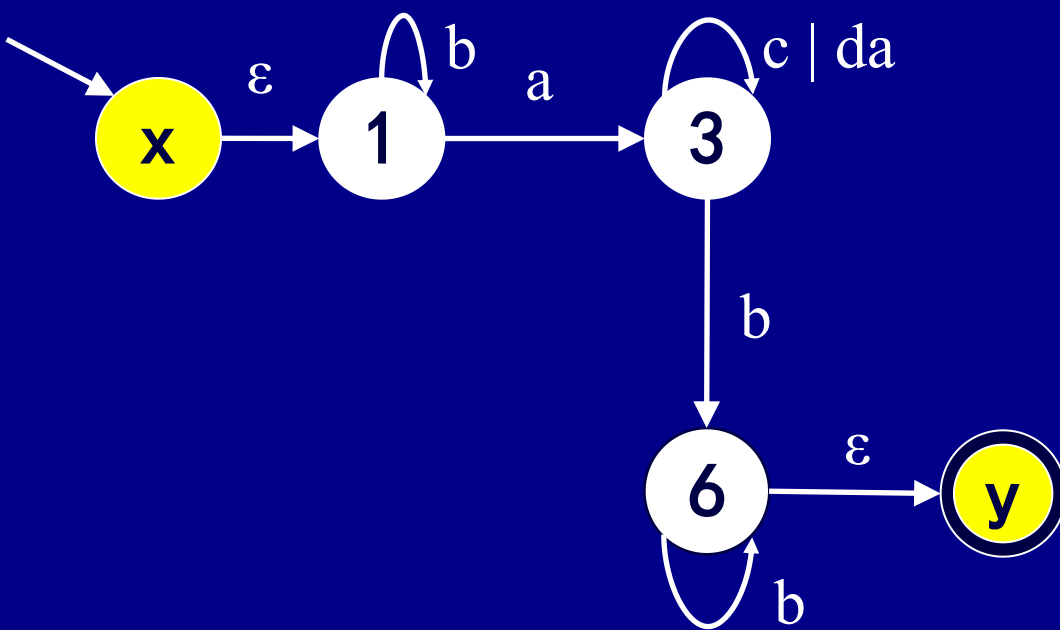


## 由NFA构造正规式

- 第二步，逐步消去 $M'$ 中的所有结点，直至只剩下 $x$ 和 $y$ 结点。在消结点过程中，逐步用正规式来标记边。消结点规则如下：



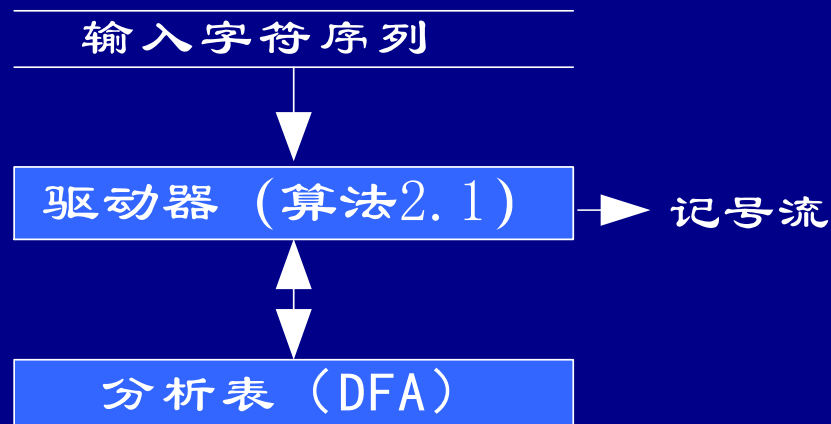






## 2.4.4 由DFA构造词法分析器

### <1> 表驱动型的词法分析器



其中，需要：

1. 有适当的数据结构存放DFA;
2. 修改算法2.1, 以适应实际输入:
  - 识别整个文件, 而不是一个记号;
  - 满足最长匹配原则。

对于输入序列:

`result := a + b`

正确的识别: `id := id + id`

错误的识别:

1. 仅识别一个: `id` (result)
2. 不满足最长匹配: `id id ...` (r或re或res ... )





## <2> 直接编码的词法分析器

在表驱动的词法分析器中，DFA是被动的，需要一个驱动器来模拟DFA的行为，以实现输入序列的分析。

直接编码的词法分析器，将DFA和DFA识别输入序列的过程合并在一起，直接用程序代码模拟DFA识别输入序列的过程。

问题：

如何用程序模拟DFA识别输入序列的过程？即如何用程序模拟DFA的状态和它的状态转移？

1. 状态和状态转移与语句的对应关系

- ① 初态→程序的开始；
- ② 终态→程序的结束（不同终态return不同记号）；
- ③ 状态转移→分情况或者条件语句（case/if）；
- ④ 环→循环语句（loop）；
- ⑤ return满足最长匹配原则。



## 2. 识别 $(a|b)^*abb$ 的程序框架 <2> 直接编码的词法分析器 (续1)

```
void main() { char buf[]="abba#", *ptr=buf;
  while (*ptr!='#')
  {
```

```
    10: while (*ptr=='b') ptr++;           // state 0
```

```
    11: while (*ptr=='a') ptr++;           // state 1
```

```
    switch (*ptr)
```

```
    { case 'a': goto 11;
```

```
      case 'b': ptr++;
```

```
        switch (*ptr)           // state 2
```

```
        { case 'a': goto 11;
```

```
          case 'b': ptr++;
```

```
            switch (*ptr) // state3
```

```
            { case 'a': goto 11;
```

```
              case 'b': goto 10;
```

```
              case '#': cout<<"yes"<<endl;
```

```
                      return;
```

```
              default: break;
```

```
            }
```

```
        default: break;
```

```
      }
```

```
    default: break;
```

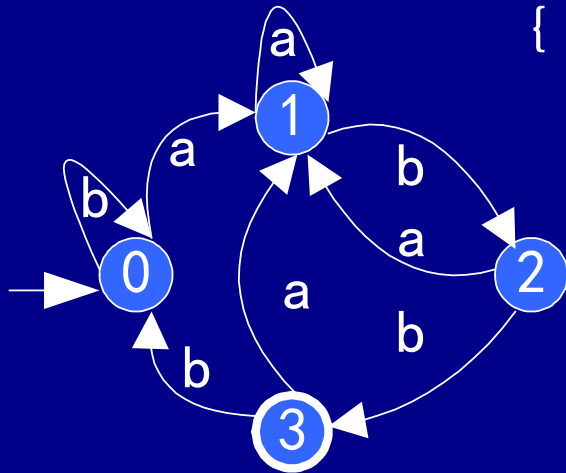
```
  }
```

```
  break;           // 遇到非法字符
```

```
}
```

```
cout << "no" << endl;
```

```
} // 看实例运行
```





### <3> 两类分析器的比较

|          | 表驱动  | 直接编码 |
|----------|------|------|
| 分析器的速度   | 慢    | 快    |
| 程序与模式的关系 | 无关   | 有关   |
| 分析器的规模   | 较大   | 较小   |
| 适合的编写方法  | 工具生成 | 手工编写 |

## 2.4.5 词法分析器生成器简介

<1> 构造词法分析器的全过程均有算法：

正规式—NFA—DFA—最小化DFA—词法分析器（分析表）

<2> LEX的基本结构

根据正规式构造的分析表+驱动器框架（不变的）

<3> 利用LEX构造词法分析器的关键

① 用LEX提供的正规式集合设计记号的模式；

② 用LEX提供的语义支持识别记号或指出输入中的错误。



## 2.5 本章小结

词法分析的两个重要环节：规定所有合法输入 + 识别合法输入  
重要内容：

<1> 记号、模式与单词

<2> 记号的说明：模式的形式化描述—正规式

<3> 记号的识别：有限自动机

NFA：与正规式有对应关系，易于构造，状态数少；

DFA：确定性便于记号识别，不易构造，状态数可能多；

记号识别的方法：

a. 模拟DFA；

b. 模拟NFA（特殊情况下）：需要动态计算状态子集。



### <4> 从正规式到词法分析器 (等价变换的过程)

- 正规式描述模式
- 由正规式构造NFA
- NFA的确定化 (子集法: smove,  $\epsilon$ -闭包)
- DFA的最小化 (可区分概念)
- 词法分析器: 表驱动 (自动生成) 与直接编码 (手工编写)